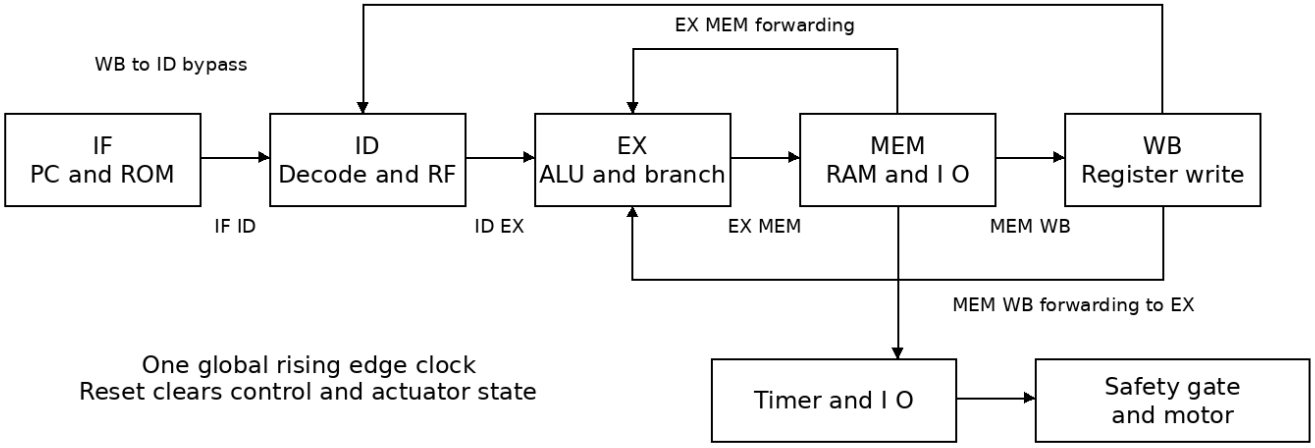
Processor architecture



Datapath and pipeline banks

A single-issue, in-order IF → ID → EX → MEM → WB pipeline executes 16-bit instructions on 8-bit operands. ROM is 256 × 16, RAM is 128 × 8, and R0–R7 are eight individual writable registers. The ALU, selection, comparators, decoders and incrementers use basic gates and custom subcircuits.

Bank	Stored fields
IF/ID	Instruction 16, PC 8, valid 1
ID/EX	PC, A, B, immediate and target (8 each); Rd/Rs (3 each); ALU_OP 3; WB_SEL 2; valid, register-write, memory-read, memory-write, immediate-select, subtract, branch, BNE, jump, HALT, use-A, use-B and compare controls
EX/MEM	ALU/address result, executed result and store data (8 each); Rd 3; WB_SEL 2; valid, register-write, memory-read and memory-write
MEM/WB	Memory data and executed result (8 each); Rd 3; WB_SEL 2; valid and register-write

WB_SEL=1 chooses load data. Other used selections choose the executed EX result, including immediate and compare results. ALU_OP=0 selects add/subtract, 1 selects AND and 6 selects operand B. LD/ST use forwarded Rs as address; ST uses forwarded old Rd as data.

Forwarding and pipeline control

For each used EX source, a valid writing EX/MEM entry with the same register number supplies the youngest ready result, unless it is a load. Otherwise a matching valid MEM/WB writer supplies WB data; otherwise the ID/EX operand is used. The EX/MEM selection is the outer mux, so it wins when both ready writers match. Load-use interlocking prevents a valid consumer from using an unavailable EX/MEM load result.

Register-file read bypass compares both ID read addresses with the current valid WB destination. On a match, ID sees WB data in that same cycle, before the next register edge. The eight stored registers and original read muxes remain unchanged.

Event	PC and IF/ID	ID/EX	Older stages
Normal	Advance	Capture decoded instruction	Advance
Load-use	Hold PC and IF/ID for one edge	Insert invalid bubble	Advance; load reaches MEM
Taken branch or jump	PC takes absolute target; IF/ID invalid	Invalidate younger entry	Advance normally
Valid HALT in EX	Sticky stop; IF/ID invalid until reset	Invalidate younger entry	Drain normally
Reset	PC=0; invalid IF/ID	Invalid	Invalidate EX/MEM and MEM/WB

Redirect and flush override a simultaneous stall. The no-stall branch penalty is two younger instructions. The hazard unit checks only the source-use bits for the decoded instruction. A load immediately before ADD or either ST source inserts exactly one bubble. Stores already in MEM advance rather than being held, so they commit once.

Register writes require WB valid and register-write. RAM and peripheral writes require MEM valid and memory-write, plus the correct address select. Invalid entries may retain unused data/control bits internally, but cannot change architectural state. Separate instruction and data paths avoid an instruction/data structural conflict.

Timer and safety behavior

The timer receives the global clock independently of CPU enables. If enabled and not clearing, COUNT increments modulo 256. The incremented value is compared to nonzero LIMIT, and equality sets sticky MATCH. CTRL bit 1 clears COUNT and MATCH with priority; CTRL bit 0 stores enable. LIMIT writes alone do not set MATCH. A disabled timer holds count; reset clears COUNT, LIMIT, MATCH and enable.

ActualMotorEnable = CPU_MOTOR_EN AND DOOR_CLOSED AND NOT TRIP. Opening the door immediately disables the physical output through the combinational door gate. At a rising edge, an active request with an open door sets the registered TRIP inhibit. Closing the door cannot clear it.

A previous-START register detects a sampled low-to-high transition. Safe rearm requires that fresh transition, a closed door and the CPU motor request already off. A held START, an open door or an active motor request cannot rearm. Reset clears both safety state and actuator requests. This is an educational digital interlock, not a certified appliance safety system.

Washing sequence

Parameter	Mode 0	Mode 1
Fill target	100	150
Wash direction 0 limit	5 edges	8 edges
Wash direction 1 limit	5 edges	8 edges
Spin limit	12 edges	16 edges
Drain completion	Water <= 10	Water <= 10

Software waits for START and a closed door, snapshots packed MODE, and then requests the door lock. It fills to unsigned water >= target, alternates motor direction for two timer intervals, drains to water <= 10, and rearms the timer for spin. It finally stops requests, unlocks, sets DISPLAY=01 and BUZZER=1, then halts. Mode bit 0 selects the parameter set; modes 2 and 3 alias modes 0 and 1.

The program initializes every register before use and does not depend on initial RAM. Temperature and RPM are readable but are not regulated. If safety trips, hardware inhibits the motor while software continues its sequence; a software abort/restart policy is outside this core program. Reset is required to start another completed wash.